ALMOX 3.0

Controle de estoque com MySQL • Documentação funcional, técnica e de testes • 09/10/2026

Turma: _____ Grupo: _____ Professor: _____
Integrantes: _____

1. Entrega e escopo

Esta documentação corresponde aos arquivos da versão 3.0.0. O sistema utiliza React e TypeScript no navegador, Node.js no servidor e MySQL/InnoDB para persistência. O ZIP contém código, interface compilada, testes e evidências; os scripts do banco são entregues separadamente em texto.

Arquivo	Finalidade
Almox_Site_MySQL.zip	Projeto executável, testes e documentação desta versão.
Almox_Atualizar_V2_Para_V3.txt	Atualização única da versão anterior. Remove produtos e movimentos antigos de almox e almox_testes e cria o novo modelo.
Almox_Banco_MySQL.txt	Instalação nova em bases sem tabelas anteriores. Não executar após o script de atualização.
LEIA-ME.txt	Passo a passo de instalação, .env, acesso inicial e execução de testes.

As duas bases começam sem produtos e sem funcionários fictícios. Existe somente o administrador solicitado: admin@almox.est / #adm321. Não foram inventados CPF, telefone ou nascimento para essa conta de sistema. Todos os funcionários comuns precisam preencher os dados completos.

2. Perfis e acesso

Perfil	Permissões
Visitante	Somente login e autocadastro. Consultas e alterações do estoque exigem sessão válida no servidor.
Funcionário	Consultar, cadastrar, editar, excluir e movimentar produtos; consultar histórico e reverter edições compatíveis de produtos.
Administrador	Todas as funções de estoque, consulta e edição de funcionários, exclusão de funcionários e histórico administrativo com reversão.

Cargo é uma descrição profissional, não uma permissão. Todo autocadastro recebe perfil funcionario. A conta principal não pode ser alterada ou excluída pela interface. Não há promoção de outros usuários para administrador nesta versão.

3. Cadastro, login e idade

Campo/regra	Comportamento implementado
Nome	2 a 100 caracteres; somente letras Unicode e espaços. Aceita acentos. Remove espaços repetidos nas extremidades e entre palavras. Não aceita números, hífen ou apóstrofo.
CPF	11 dígitos, validação dos dois dígitos verificadores; sequências repetidas são recusadas. Campo da interface limita a 11 dígitos. CPF único.
Telefone	10 ou 11 dígitos, incluindo DDD. Campo limita a 11.
E-mail	Domínio exato @almox.est; normalizado para minúsculas; máximo 150 caracteres e valor único. Isso valida formato, não confirma existência de caixa postal.
Nascimento	Data real, idade entre 16 e 120 anos. No aniversário de 16 anos o cadastro passa a ser permitido.
Cargo	2 a 80 caracteres. Não concede privilégios.
Senha	6 a 128 caracteres. Hash scrypt com salt aleatório de 16 bytes; senha não aparece na API nem nos históricos.
Login e sessão	Sessão de 8 horas em cookie HttpOnly, SameSite=Strict. Token aleatório; somente hash do token é persistido. Sair remove a sessão. Excluir funcionário revoga suas sessões pelo banco.
Tentativas de login	10 falhas por endereço IP em janela de 15 minutos geram bloqueio temporário. Contador em memória, reiniciado com o servidor.

A idade é persistida em usuarios.idade e recalculada a partir de nascimento. O servidor sincroniza ao consultar sessões/equipe e a cada minuto. O evento atualizar_idades atualiza diariamente às 03:00 UTC, equivalentes à meia-noite de Brasília, quando event_scheduler está ON. Consulte LEIA-ME para ativar e manter essa configuração após reiniciar o MySQL. Com banco desligado, a próxima execução/acesso corrige a idade. Para nascimento em 29/02, a idade aumenta em 01/03 nos anos sem 29/02.

A atualização automática da idade é um dado derivado, não uma edição manual; não gera um evento administrativo a cada aniversário. Alterar nascimento pelo administrador é auditado e recalcula a idade. A idade do cadastro fica registrada como fotografia do momento.

4. Produtos, preços e movimentações

Funcionalidade	Regra e resultado
Novo produto	Nome com 2 a 100 caracteres; preço maior que zero, máximo R$1.000.000,00; saldo inicial inteiro entre 0 e 1.000.000. Cadastro e saldo inicial geram um registro de cadastro no histórico. Nomes duplicados são permitidos.
Preço	O campo tem máscara monetária: digitar 2000 apresenta R$20,00. Banco: produtos.preco DECIMAL(10,2), em reais, valor 20.00. Nunca centavos. A view produtos_formatados retorna a apresentação R$20,00 sem prejudicar cálculos.
Consulta	Busca por trecho do nome, sem diferenciar maiúsculas. Exibe nome legível, preço, saldo, disponibilidade e ações. Não exibe UUID como identificação do produto.
Painel	Valor total = soma dos preços vezes saldos; quantidade de produtos, total de unidades e produtos com saldo zero. Atualiza após operações e pelo botão Atualizar. Não possui sincronização instantânea entre navegadores.
Editar	Altera nome e preço. Saldo não é editado nesse formulário; deve ser alterado por entrada ou saída. Exige confirmação e grava antes/depois.
Entrada e saída	Quantidade inteira entre 1 e 1.000.000. Saída não ultrapassa saldo; saída igual ao saldo resulta em zero. Entrada não deixa saldo acima de 1.000.000. Observação opcional de até 200 caracteres.
Excluir produto	Exclusão física do cadastro após confirmação. O histórico mantém nome, responsável, data/hora e dados anteriores, inclusive saldo e preço. Não cria uma saída artificial do saldo remanescente.
Reversão de edição	Restaura nome/preço do produto ou dados do funcionário e gera novo evento. Não desfaz movimentos de estoque. Não restaura registros excluídos. Edição já revertida não pode ser revertida novamente.

Para excluir, a mensagem é: “Tem certeza que deseja realizar essa exclusão? Não será possível reverter essa ação!”. Para editar, a mensagem informa que as alterações poderão ser revertidas pelo histórico. Cancelar não envia uma alteração ao banco.

A reversão confere se os campos atuais ainda correspondem ao resultado da edição original. Se outra edição mudou os campos, bloqueia a reversão antiga para evitar sobrescrever trabalho posterior. É necessário desfazer primeiro a edição posterior. A reversão do produto conserva o saldo atual.

5. Administração e histórico

Funcionários: o administrador consulta nome, idade, CPF, telefone, e-mail, nascimento e cargo. A edição permite mudar esses dados pessoais/profissionais, mantendo validações e unicidade. Não permite editar a senha ou o perfil de acesso. Não há recuperação de senha ou edição de perfil pelo próprio funcionário nesta versão.

O histórico de estoque contém cadastro, entrada, saída, edição, exclusão e reversão de produtos. O histórico administrativo contém autocadastro, edição, exclusão e reversão de funcionários. Cada evento preserva o nome do alvo e do autor no momento da operação, tipo, data/hora e fotografia JSON antes/depois. O cadastro identifica o próprio funcionário como autor da solicitação de criação.

A interface apresenta campos modificados, valor anterior e novo, ordenados pelos eventos mais recentes. Datas de eventos são gravadas em UTC e exibidas no fuso America/Sao_Paulo. Dados pessoais dos funcionários e seu histórico completo ficam restritos à API/tela administrativa. Histórico do estoque é acessível à equipe autenticada. Não existe botão para apagar o histórico.

6. Transação: saldo e histórico juntos

Exemplo: há 10 mouses e uma saída de 3 foi solicitada. O servidor abre uma transação na mesma conexão, bloqueia a linha do produto com SELECT ... FOR UPDATE, valida a saída, muda o saldo para 7 e insere o histórico. Somente depois das duas gravações executa COMMIT. Se qualquer etapa falhar, executa ROLLBACK e o saldo continua 10, sem novo histórico. Isso é atomicidade: a operação fica inteira ou não fica.

O bloqueio da linha também impede duas saídas simultâneas de usarem o mesmo saldo antigo. O teste TI03 solicita duas saídas de 7 a partir de 10: apenas uma é aprovada; a outra encontra saldo 3 e é recusada. Exclusões, cadastros, edições e reversões também agrupam alteração e auditoria em uma transação.

A garantia depende de usar as operações do servidor e tabelas InnoDB. Alterações manuais pelo Workbench não passam automaticamente pela lógica do site e podem não gerar histórico. A validação de CPF, formato completo do nome, idade mínima e permissões é feita no servidor; as restrições SQL complementam essa validação.

7. Estrutura do código

Caminho	Responsabilidade
app/page.tsx / globals.css	Telas, formulários, confirmação, máscaras, filtros e estilos responsivos.
lib/rules.ts	Regras isoladas de preço, quantidade, saldo, CPF, idade e funcionário.
server/index.mjs	HTTP, sessão, autorização das rotas, respostas e arquivos compilados.
server/service.mjs / db.mjs	Operações, auditoria, consultas, bloqueios, pool e transações.
server/auth.mjs	Hash de senhas, tokens e data de referência de Brasília.
tests/ / evidencias/	Testes automatizados e capturas/saídas obtidas.

8. Modelo de dados e relacionamentos

Tabela	Campos e tipos principais
usuarios	id CHAR(36) PK; nome VARCHAR(100); cpf CHAR(11) UNIQUE; telefone VARCHAR(11); email VARCHAR(150) UNIQUE; nascimento DATE; idade SMALLINT; cargo VARCHAR(80); perfil ENUM; conta_sistema BOOLEAN; senha_hash VARCHAR(200); criado_em DATETIME.
produtos	id CHAR(36) PK; nome VARCHAR(100); preco DECIMAL(10,2); quantidade INT; criado_em DATETIME. CHECKs: nome 2..100, preço >0 e <=1.000.000, quantidade 0..1.000.000.
sessoes	token_hash CHAR(64) PK; usuario_id CHAR(36) FK; expira_em DATETIME. FK ON DELETE CASCADE revoga as sessões ao excluir usuário.
movimentacoes	sequencia BIGINT UNSIGNED AUTO_INCREMENT PK; id CHAR(36) UNIQUE; alvo_id e usuario_id CHAR(36) FK; alvo_nome e usuario_nome VARCHAR(100); acao ENUM; antes/depois JSON; quantidade INT; observacao VARCHAR(200); revertido BOOLEAN; criado_em DATETIME.
auditoria_admin	Mesmos campos do histórico de estoque; alvo_id referencia usuarios em vez de produtos.
produtos_formatados	VIEW: id, nome, preco, preco_formatado (texto com R$ e vírgula), quantidade. Não duplica o preço em outra tabela.

Um produto possui muitos eventos de estoque; um usuário pode ser autor de muitos eventos e ter muitas sessões. As duas FKs dos históricos usam ON DELETE SET NULL: ao excluir o cadastro, as chaves ficam nulas, mas as cópias do nome e das características permanecem. As tabelas usam InnoDB e as bases utf8mb4_unicode_ci.

As regras SQL de usuários incluem e-mail com sufixo @almox.est, CPF numérico de 11 dígitos e telefone numérico de 10/11 dígitos, além da exigência de dados/idade para contas comuns. Não substituem a validação mais completa do servidor. O valor de idade salvo não é editável na interface.

9. API

Rota	Método, acesso e função
/api/cadastro	POST público: cria somente funcionário, com auditoria.
/api/login / /api/logout	POST: autentica ou encerra a sessão.
/api/me	GET autenticado: dados do usuário atual.
/api/estoque	GET autenticado: produtos e histórico. POST: action produto, editar, movimentar, excluir ou reverter.
/api/admin	GET/POST somente admin: equipe/histórico; action editar, excluir ou reverter.

JSON limitado a 16 KiB; gravações verificam Content-Type e origem quando informada pelo navegador. Consultas SQL usam parâmetros. Erros de entrada: 400; sem sessão: 401; sem permissão: 403; inexistente: 404; conflito: 409; excesso de tentativas: 429; falha de banco: 503. Interface não considera sucesso antes da resposta do servidor.

10. Pesquisa: níveis de teste

Nível	Objetivo, objeto e exemplo deste sistema
Unidade	Verificar uma função pequena isoladamente. Ex.: price("R$20,00") devolve "20.00" e price("0") falha. Não usa banco real.
Integração	Verificar colaboração do serviço com o MySQL, FKs, persistência e transação. Ex.: INSERT do histórico falha e UPDATE do saldo é desfeito.
Sistema	Verificar o produto completo pela interface. Ex.: usuário entra, cadastra produto, tenta saída excessiva, corrige e consulta histórico.
Aceitação	O grupo cliente verifica se a solução atende critérios de uso combinados. Ex.: conseguir excluir produto e ainda identificar o responsável pela exclusão.

Unidade isola a regra; integração exercita fronteiras e comunicação. Sistema verifica o funcionamento completo; aceitação verifica se ele atende à necessidade do cliente. Uma automação de interface não substitui a aprovação pelo outro grupo exigida na atividade.

Ferramentas usadas: node:test e node:assert para unidade e integração; mysql2 para conexão real; Playwright/Chromium para sistema. Aceitação usa a interface e o formulário a seguir. Os dados de teste são gerados por scripts somente em almox_testes; os executores recusam nomes que não terminem em _testes. Os testes de integração/sistema limpam essa base, portanto não aponte para uma base real.

11. Execução, evidências e falhas

Nesta entrega: 5 testes de unidade, 4 de integração e 4 de sistema aprovados. MySQL real 8.0.46; Node 24.19; Chromium headless 134. O executável do navegador foi informado por CHROMIUM_PATH no ambiente de validação. Na instalação normal use npx playwright install chromium e npm run test:system. A aceitação pelo outro grupo está pendente, sem aprovação inventada.

Uma execução inicial dos testes de interface falhou porque o seletor automatizado não localizava o campo Tipo. A correção usou o papel acessível combobox; a nova execução passou. O arquivo evidencias/execucao-inicial-interface.txt preserva a falha. Outra tentativa foi bloqueada pelo navegador completo no ambiente; a execução com navegador headless funcionou, preservada em execucao-bloqueada-navegador.txt. Não se tratou de perda de dados ou falha na regra de estoque.

Resultado final: evidencias-testes.txt. Capturas: TS01-login.png, TS02-saida-invalida.png, TS03-historico.png, TS04-admin.png e TS04-mobile.png. A prova do rollback é o teste automatizado TI02: trigger de teste provoca uma falha real; consulta confirma saldo 10 e apenas o cadastro original no histórico. A trigger é removida em finally.

Os casos seguintes usam os campos exigidos na atividade. Para repetir testes, anexe nova evidência, data e responsável sem apagar o registro anterior. Datas e tempos nas evidências são da execução de 09/10/2026; não representam execução no computador da escola.

TU01 | Unidade

Preço em reais e limites

Campo	Registro
Identificação	TU01
Nível	Unidade
Objetivo	Preço em reais e limites
Preparação	Nenhum banco; carregar price de lib/rules.ts.
Procedimento	Executar npm test. Validar R$20,00, 0,01, zero, negativo, três casas, texto e acima do limite.
Resultado esperado	20.00 e 0.01 aceitos; demais valores inválidos recusados.
Resultado obtido	Aprovado; todas as asserções passaram.
Situação	Aprovado
Evidência	evidencias-testes.txt / tests/unit.test.mjs

TU02 | Unidade

Quantidade e saída no limite

Campo	Registro
Identificação	TU02
Nível	Unidade
Objetivo	Quantidade e saída no limite
Preparação	Saldo fictício 10 em memória.
Procedimento	Testar saída 10, saída 11, quantidade zero, fracionária e zero permitido apenas como saldo inicial.
Resultado esperado	Saída 10 resulta 0; excesso/zero/fracionária recusados.
Resultado obtido	Aprovado; limites confirmados.
Situação	Aprovado
Evidência	evidencias-testes.txt / tests/unit.test.mjs

TU03 | Unidade

Idade mínima, CPF e dados pessoais

Campo	Registro
Identificação	TU03
Nível	Unidade
Objetivo	Idade mínima, CPF e dados pessoais
Preparação	Data de referência fixa 09/10/2026; nascimento 09/10/2010; CPF fictício 52998224725.
Procedimento	Testar véspera/aniversário de 16 anos, 29/02, CPF repetido/longo, nome numérico, domínio errado, telefone curto e data inexistente.
Resultado esperado	Cadastro permitido ao completar 16; demais dados inválidos recusados; 29/02 muda idade em 01/03 no ano não bissexto.
Resultado obtido	Aprovado; todas as asserções passaram.
Situação	Aprovado
Evidência	evidencias-testes.txt / tests/unit.test.mjs

TU04 | Unidade

Rollback no controlador de transação

Campo	Registro
Identificação	TU04
Nível	Unidade
Objetivo	Rollback no controlador de transação
Preparação	Conexão simulada, sem banco.
Procedimento	Provocar erro no trabalho da transação; observar begin, rollback e release.
Resultado esperado	Não executar commit; liberar conexão após rollback.
Resultado obtido	Aprovado; sequência esperada confirmada.
Situação	Aprovado
Evidência	evidencias-testes.txt / tests/service.test.mjs

TU05 | Unidade

Hash da senha

Campo	Registro
Identificação	TU05
Nível	Unidade
Objetivo	Hash da senha
Preparação	Senha de teste #adm321.
Procedimento	Gerar hash, comparar com senha correta e incorreta.
Resultado esperado	Hash difere do texto puro; correta aceita e errada recusada.
Resultado obtido	Aprovado.
Situação	Aprovado
Evidência	evidencias-testes.txt / tests/service.test.mjs

TI01 | Integração

Persistência, reais, FK e sessão

Campo	Registro
Identificação	TI01
Nível	Integração
Objetivo	Persistência, reais, FK e sessão
Preparação	MySQL em base exclusiva _testes; administrador inicial.
Procedimento	Executar npm run test:integration. Cadastrar produto com 10 unidades/R$20,00; consultar; tentar FK inválida; autenticar.
Resultado esperado	Preço 20.00 persistido; nome no evento; FK inválida recusada; sessão identifica admin.
Resultado obtido	Aprovado no MySQL real.
Situação	Aprovado
Evidência	evidencias-testes.txt / tests/mysql.test.mjs

TI02 | Integração

Atomicidade entre saldo e histórico

Campo	Registro
Identificação	TI02
Nível	Integração
Objetivo	Atomicidade entre saldo e histórico
Preparação	Produto do TI01: 10 unidades; trigger temporária rejeita INSERT em movimentacoes.
Procedimento	Solicitar saída 3. Após erro, consultar produto e histórico. Remover trigger ao final.
Resultado esperado	Saldo continua 10; histórico continua com somente o cadastro inicial.
Resultado obtido	Aprovado; falha real de INSERT desfez UPDATE.
Situação	Aprovado
Evidência	evidencias-testes.txt / tests/mysql.test.mjs

TI03 | Integração

Concorrência, reversão e exclusão

Campo	Registro
Identificação	TI03
Nível	Integração
Objetivo	Concorrência, reversão e exclusão
Preparação	Produto com saldo 10, sem trigger.
Procedimento	Enviar duas saídas de 7 simultâneas; editar nome/preço; reverter; excluir; consultar eventos.
Resultado esperado	Só uma saída aceita e saldo 3; reversão restaura nome/preço; exclusão remove produto e mantém nome/autor nos eventos.
Resultado obtido	Aprovado.
Situação	Aprovado
Evidência	evidencias-testes.txt / tests/mysql.test.mjs

TI04 | Integração

Funcionário, permissões, idade e auditoria

Campo	Registro
Identificação	TI04
Nível	Integração
Objetivo	Funcionário, permissões, idade e auditoria
Preparação	Cadastro fictício Maria Silva, CPF válido de teste, nascimento 01/01/2000.
Procedimento	Cadastrar e autenticar; tentar operação admin como funcionário; editar/reverter como admin; defasar idade no teste e sincronizar; excluir funcionário.
Resultado esperado	Acesso comum recusado; dados e auditoria corretos; idade corrigida; sessão revogada; histórico preservado; admin principal protegido.
Resultado obtido	Aprovado.
Situação	Aprovado
Evidência	evidencias-testes.txt / tests/mysql.test.mjs

TS01 | Sistema

Acesso protegido e cadastro pela interface

Campo	Registro
Identificação	TS01
Nível	Sistema
Objetivo	Acesso protegido e cadastro pela interface
Preparação	Base _testes limpa; servidor automático na porta 3101; navegador sem sessão.
Procedimento	Executar npm run test:system. Abrir site; tentar cadastro com gmail; corrigir domínio; entrar como funcionário; tentar API admin.
Resultado esperado	Sem login estoque responde 401; domínio incorreto recusado; cadastro/login válidos; admin retorna 403 para funcionário.
Resultado obtido	Aprovado no navegador e API.
Situação	Aprovado
Evidência	evidencias/TS01-login.png; evidencias-testes.txt

TS02 | Sistema

Preço formatado, saída inválida e persistência

Campo	Registro
Identificação	TS02
Nível	Sistema
Objetivo	Preço formatado, saída inválida e persistência
Preparação	Funcionário autenticado pelo TS01.
Procedimento	Cadastrar Mouse de teste com preço digitado 2000 e saldo 10; tentar saída 11; corrigir para 10; recarregar página.
Resultado esperado	Campo/tabela R$20,00, banco 20.00; excesso recusado; saldo final 0 persiste após recarregar.
Resultado obtido	Aprovado após corrigir seletor do teste; execução inicial preservada.
Situação	Aprovado
Evidência	evidencias/TS02-saida-invalida.png; execucao-inicial-interface.txt; evidencias-testes.txt

TS03 | Sistema

Editar, reverter e excluir pela interface

Campo	Registro
Identificação	TS03
Nível	Sistema
Objetivo	Editar, reverter e excluir pela interface
Preparação	Produto do TS02; funcionário autenticado.
Procedimento	Editar nome e confirmar; reverter no histórico; excluir e confirmar; consultar histórico.
Resultado esperado	Nome restaurado pela reversão; produto excluído; histórico exibe Mouse de teste, Maria Teste, data/hora e exclusão.
Resultado obtido	Aprovado.
Situação	Aprovado
Evidência	evidencias/TS03-historico.png; evidencias-testes.txt

TS04 | Sistema

Administração com auditoria

Campo	Registro
Identificação	TS04
Nível	Sistema
Objetivo	Administração com auditoria
Preparação	Maria Teste cadastrada pelo TS01.
Procedimento	Sair, entrar com admin; editar nome da funcionária, reverter, excluir e consultar histórico administrativo; revisar tela em 390 px.
Resultado esperado	Ações confirmadas; antes/depois e responsável visíveis; funcionária excluída; sem erros JavaScript.
Resultado obtido	Aprovado; capturas desktop/mobile revisadas.
Situação	Aprovado
Evidência	evidencias/TS04-admin.png; TS04-mobile.png; evidencias-testes.txt

TA01 | Aceitação

Cliente consegue controlar estoque em reais

Campo	Registro
Identificação	TA01
Nível	Aceitação
Objetivo	Cliente consegue controlar estoque em reais
Preparação	Grupo cliente; base de testes; conta de funcionário e produto novo com saldo 10.
Procedimento	Cliente cadastra produto a R$20,00, retira 10, tenta retirar mais 1 e reabre o sistema.
Resultado esperado	Preço legível em reais; saldo 0 persistente; saída extra recusada e sem evento indevido.
Resultado obtido	Não executado pelo grupo cliente. Preencher resultado, responsável, data e evidência.
Situação	Bloqueado - aguarda execução pelo grupo cliente
Evidência	Anexar captura da tabela, erro e histórico; grupo cliente registra aprovação.

Responsável / grupo cliente: _____

Data e horário: _____

Resultado obtido: _____

_____

Situação após execução: [ ] Aprovado [ ] Reprovado [ ] Bloqueado

Evidência e observações: _____

_____

TA02 | Aceitação

Cliente identifica o que mudou e quem excluiu

Campo	Registro
Identificação	TA02
Nível	Aceitação
Objetivo	Cliente identifica o que mudou e quem excluiu
Preparação	Grupo cliente; produto de teste cadastrado.
Procedimento	Cliente edita nome/preço, confirma, confere antes/depois, reverte, exclui e consulta histórico.
Resultado esperado	Reversão funciona; exclusão pede confirmação irreversível; histórico mantém nome do produto, responsável e data/hora.
Resultado obtido	Não executado pelo grupo cliente. Preencher resultado, responsável, data e evidência.
Situação	Bloqueado - aguarda execução pelo grupo cliente
Evidência	Anexar capturas da confirmação e histórico.

Responsável / grupo cliente: _____

Data e horário: _____

Resultado obtido: _____
_____

Situação após execução: [ ] Aprovado [ ] Reprovado [ ] Bloqueado

Evidência e observações: _____
_____

TA03 | Aceitação

Admin gerencia equipe sem liberar funções a funcionários

Campo	Registro
Identificação	TA03
Nível	Aceitação
Objetivo	Admin gerencia equipe sem liberar funções a funcionários
Preparação	Grupo cliente; funcionário com idade mínima válida; acesso admin.
Procedimento	Cliente entra como funcionário e verifica ausência de gestão de equipe; admin edita/reverte dados e exclui funcionário; tentar novo login excluído.
Resultado esperado	Funções administrativas restritas; antes/depois auditados; exclusão confirmada; login do excluído recusado.
Resultado obtido	Não executado pelo grupo cliente. Preencher resultado, responsável, data e evidência.
Situação	Bloqueado - aguarda execução pelo grupo cliente
Evidência	Anexar histórico administrativo e tentativa de acesso recusada.

Responsável / grupo cliente: _____

Data e horário: _____

Resultado obtido: _____

_____

Situação após execução: [ ] Aprovado [ ] Reprovado [ ] Bloqueado

Evidência e observações: _____

_____

12. Organização do grupo e rastreabilidade

Tarefa	Responsável a definir / entrega
Interface e acessibilidade	_____ / formulários, fluxos e confirmação.
Servidor e segurança	_____ / login, permissões e validações.
Banco e integração	_____ / modelo SQL, FKs e transações.
Testes e documentação	_____ / execução, tabelas, evidências e correções.
Cliente de outro grupo	_____ / executar TA01, TA02 e TA03.

Requisito	Testes relacionados
Preço positivo e reais	TU01, TI01, TS02, TA01
Quantidade positiva e saldo suficiente	TU02, TI02, TI03, TS02, TA01
Transação e persistência	TU04, TI01, TI02, TI03, TS02
Cadastro e acesso por perfil	TU03, TU05, TI04, TS01, TS04, TA03
Auditoria, exclusão e reversão	TI03, TI04, TS03, TS04, TA02, TA03

13. Fontes consultadas

ISTQB. CTFL Syllabus 4.0.1, seção 2.2: níveis de teste. https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

MySQL. InnoDB: autocommit, Commit, and Rollback. https://dev.mysql.com/doc/refman/8.4/en/innodb-autocommit-commit-rollback.html

MySQL. Cálculos de idade com TIMESTAMPDIFF. https://dev.mysql.com/doc/refman/8.4/en/date-calculations.html

MySQL. Event Scheduler. https://dev.mysql.com/doc/refman/8.0/en/event-scheduler.html

Node.js. Test runner e crypto.scrypt. https://nodejs.org/api/test.html e https://nodejs.org/api/crypto.html

Playwright. Testes de navegador e localizadores. https://playwright.dev/docs/locators

Consulta em 09/10/2026. Exemplos, regras e resultados desta documentação foram derivados do código e das execuções da versão entregue. A aprovação de aceitação deve ser preenchida pelo grupo cliente.